Studio Nx

백엔드 152 — create_reset_token()

2026-10-14

백엔드 152 — `create_reset_token()` (재설정 토큰 저장)

항목	내용
파일	`studionx/backend/app/services/password_reset_service.py` (새 파일), 설정 `core/config.py`의 `password_reset_minutes`
작업	P1-28 비밀번호 찾기 (FR-118): 세 번째 함수
작성	Claude (AI), 2026-10-14, 사용자 검토
이전	151 `generate_reset_token()`
시험	`tests/test_password_reset_service.py` 7개
다음	`request_password_reset()`: 이메일로 계정을 찾고, 횟수 제한을 검사하고, 이 함수를 부르고, 링크를 로그에 출력

1. 하는 일

회원 한 명의 재설정 토큰을 **새로 만들어 저장**하고, 메일 링크에 쓸 **원문**을 돌려줍니다.

순서	일
1	이 회원의 이전 토큰 중 아직 쓰지 않은 것을 모두 **사용 처리**(`used_at`)합니다
2	`generate_reset_token()`으로 (원문, 해시)를 만들고 **해시만** 저장합니다. 만료는 지금 + 30분입니다
3	`commit`하고 원문을 돌려줍니다. 호출한 쪽이 메일 링크에만 쓰고 저장하지 않습니다

2. 코드

```
def create_reset_token(db: Session, user: User) -> str:
    now = datetime.now(UTC).replace(tzinfo=None)
    expires_at = now + timedelta(minutes=get_settings().password_reset_minutes)

    db.execute(
        update(PasswordResetToken)
        .where(PasswordResetToken.user_id == user.user_id, PasswordResetToken.used_at.is_(None))
        .values(used_at=now)
    )
    token, token_hash = generate_reset_token()
    db.add(PasswordResetToken(user_id=user.user_id, token_hash=token_hash, expires_at=expires_at))
    db.commit()
    return token
```

3. 왜 이렇게

선택	이유
이전 토큰을 사용 처리	다시 요청하면 이전 메일의 링크는 못 쓰게 됩니다. 살아 있는 링크가 항상 하나뿐이라, 오래된 메일이 뒤늦게 악용될 여지가 줄어듭니다
`used_at IS NULL`인 것만 갱신	이미 쓴 토큰의 사용 시각을 덮어쓰지 않습니다(기록 보존)

선택	이유
삭제하지 않고 사용 처리	만료와 사용 기록이 남아서 "이미 쓴 링크입니다"와 "만료된 링크입니다"를 구분해 안내할 수 있습니다. 쌓인 행은 정리 작업(`retention`)에서 지웁니다
한 번에 `commit`	이전 토큰 무효화와 새 토큰 저장이 함께 성공하거나 함께 실패합니다
UTC에서 시간대 정보를 뗀 시각	DB의 `DATETIME`은 시간대가 없고 서버는 UTC로 연결합니다(`session.py`). 다른 서비스와 같은 방식입니다
만료 시간을 설정값으로	`password_reset_minutes`(기본 30)로 두어 시험이나 운영에서 바꿀 수 있습니다
상태와 횟수 제한은 여기서 검사하지 않음	부르는 쪽(요청 서비스)이 먼저 거릅니다. 이 함수는 "토큰을 저장한다"는 한 가지 일만 합니다

4. 시험 (7개 통과)

확인	결과
돌려준 원문의 해시가 저장된 해시와 같고, 원문은 저장되지 않음	✓
만료가 지금 + 30분	✓
새 토큰은 사용하지 않은 상태	✓
두 번째 요청이 첫 번째를 무효로 만들고, 살아 있는 것은 하나뿐	✓
다른 회원의 토큰은 건드리지 않음	✓
이미 쓴 토큰의 사용 시각은 덮어쓰지 않음	✓
5번 요청해도 행 5개, 살아 있는 것은 1개	✓

시험을 확인하는 시험. `used_at=now` 대신 `used_at=None`으로 일부러 틀리게 바꿔 보니 "두 번째 요청이 첫 번째를 무효로 만듦"과 "살아 있는 것은 하나뿐" 두 시험이 실패했고, 원래대로 되돌리자 통과했습니다.

시험에서 고친 것. 만료 시각 시험이 전체 실행에서 한 번 실패했습니다. DB의 `DATETIME(3)`이 밀리초로 반올림해 저장해서, 시험의 기준 시각보다 1밀리초쯤 작게 읽혔기 때문입니다. 함수의 문제가 아니라 시험의 기준이 너무 빡빡했던 것이어서 1초 여유를 두었고, 이후 반복 실행에서 모두 통과했습니다.

전체 pytest 708개 통과, `ruff` 통과.